

PRACTICAL-9

Task-1: Implement a file copy utility using low-level Linux file I/O system calls (open, read, write, lseek, close). Compare its performance with standard library functions (fopen, fread, fwrite).

```
GNU nano 7.2      prac9task1.c *      Close
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/time.h>

#define BUFFER_SIZE 4096

double get_time()
{
    struct timeval tv;
    gettimeofday(&tv, NULL);
    return tv.tv_sec + tv.tv_usec / 1000000.0;
}

/* Low-level file copy using open, read, write, lseek, close */
void low_level_copy(const char *source, const char *destination)
{
    int src, dest;
    char buffer[BUFFER_SIZE];
    ssize_t bytes_read, bytes_written;

    src = open(source, O_RDONLY);

    if (src == -1)
    {
        perror("Error opening source file");
        exit(EXIT_FAILURE);
    }

    dest = open(destination, O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (dest == -1)
    {
        perror("Error opening destination file");
    }

    ^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute
```

```
GNU nano 7.2      prac9task1.c *
    }

    while ((bytes_read = read(src, buffer, BUFFER_SIZE)) > 0)
    {
        bytes_written = write(dest, buffer, bytes_read);

        if (bytes_written != bytes_read)
        {
            perror("write failed");
            close(src);
            close(dest);
            exit(EXIT_FAILURE);
        }
    }

    if (bytes_read == -1)
    {
        perror("read failed");
    }

    close(src);
    close(dest);
}

/* Standard library file copy using fopen, fread, fwrite */
void standard_copy(const char *source, const char *destination)
{
    FILE *src, *dest;
    char buffer[BUFFER_SIZE];
    size_t bytes_read;

    src = fopen(source, "rb");

    if (src == NULL)
    {
        perror("Error opening source file");
        exit(EXIT_FAILURE);
    }
    |
```

```

GNU nano 7.2      prac9task1.c *
    perror("Error opening source file");
    exit(EXIT_FAILURE);
}

dest = fopen(destination, "wb");
if (dest == NULL)
{
    perror("Error opening destination file");
    fclose(src);
    exit(EXIT_FAILURE);
}

while ((bytes_read = fread(buffer, 1, BUFFER_SIZE, src)) > 0)
{
    if (fwrite(buffer, 1, bytes_read, dest) != bytes_read)
    {
        perror("fwrite failed");
        fclose(src);
        fclose(dest);
        exit(EXIT_FAILURE);
    }
}

fclose(src);
fclose(dest);
}

int main()
{
    const char *source = "input.txt";
    const char *low_level_destination = "copy_lowlevel.txt";
    const char *standard_destination = "copy_standard.txt";

    double start, end;
    double low_level_time, standard_time;

```

```

    start = get_time();

    low_level_copy(source, low_level_destination);

    end = get_time();

    low_level_time = end - start;

    printf("\nLow-level I/O copy completed.\n");
    printf("Destination: %s\n", low_level_destination);
    printf("Time: %.9f seconds\n", low_level_time);

    /* Standard library copy */
    start = get_time();

    standard_copy(source, standard_destination);

    end = get_time();

    standard_time = end - start;

    printf("\nStandard library I/O copy completed.\n");
    printf("Destination: %s\n", standard_destination);
    printf("Time: %.9f seconds\n", standard_time);

    printf("\n===== \n");
    printf("          PERFORMANCE COMPARISON\n");
    printf("===== \n");

    printf("Low-level I/O      : %.9f seconds\n", low_level_time);
    printf("Standard library   : %.9f seconds\n", standard_time);

    printf("\nBoth copying methods completed successfully.\n");

    return 0;
}

```

```

ruhi0404@Ruhitha:~/Operating_Systems/skill12s24task1$ cd ~/Operating_Systems
ruhi0404@Ruhitha:~/Operating_Systems$ mkdir -p skill12s24task2
ruhi0404@Ruhitha:~/Operating_Systems$ cd skill12s24task2
ruhi0404@Ruhitha:~/Operating_Systems/skill12s24task2$ pwd
/home/ruhi0404/Operating_Systems/skill12s24task2
ruhi0404@Ruhitha:~/Operating_Systems/skill12s24task2$ nano demo.c
ruhi0404@Ruhitha:~/Operating_Systems/skill12s24task2$ gcc -Wall -Wextra demo.c -o demo
ruhi0404@Ruhitha:~/Operating_Systems/skill12s24task2$ ls
demo demo.c
ruhi0404@Ruhitha:~/Operating_Systems/skill12s24task2$ ./demo
=====
FINAL OS PROJECT DEMONSTRATION
=====
--- Scenario 1: Process Management ---
[Process Demo] Child process running.
[Process Demo] Child PID: 1038
[Process Demo] Parent PID: 1037
[Process Demo] Parent successfully waited for child.

--- Scenario 2: Pipeline Demonstration ---
[Pipeline Demo] Parent received: Pipeline data transferred successfully

--- Scenario 3: Signal Demonstration ---
Process PID: 1037
Send SIGINT using another terminal:
kill -SIGINT 1037
[Signal Demo] No SIGINT received during demo period.

--- Scenario 4: Feature Validation ---
[Validation] Process creation : PASS
[Validation] Parent-child control : PASS
[Validation] IPC pipeline : PASS
[Validation] Signal handling : PASS

=====
FINAL PROJECT REVIEW COMPLETE
=====
ruhi0404@Ruhitha:~/Operating_Systems/skill12s24task2$
ruhi0404@Ruhitha:~/Operating_Systems/skill12s24task2$

```

```

ruhi0404@Ruhitha:~/Operating_Systems/prac9task1$ ls -lh
total 36K
-rw-r--r-- 1 ruhi0404 ruhi0404 157 Sep 28 14:14 copy_lowlevel.txt
-rw-r--r-- 1 ruhi0404 ruhi0404 157 Sep 28 14:14 copy_standard.txt
-rw-r--r-- 1 ruhi0404 ruhi0404 157 Sep 28 14:12 input.txt
-rwxr-xr-x 1 ruhi0404 ruhi0404 17K Sep 28 14:14 prac9task1
-rw-r--r-- 1 ruhi0404 ruhi0404 3.7K Sep 28 14:14 prac9task1.c
ruhi0404@Ruhitha:~/Operating_Systems/prac9task1$ diff input.txt copy_lowlevel.txt
ruhi0404@Ruhitha:~/Operating_Systems/prac9task1$ diff input.txt copy_standard.txt
ruhi0404@Ruhitha:~/Operating_Systems/prac9task1$ strace -e trace=openat,read,write,lseek,close ./prac9task1
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
close(3) = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\220\243\2\0\0\0\0\0"..., 832) = 832
close(3) = 0
write(1, "===== "..., 45) =====
D = 45
write(1, "FILE COPY PERFORMANCE TEST"..., 34) FILE COPY PERFORMAN
D = 34
write(1, "===== "..., 45) =====
D = 45
write(1, "\nSource file: input.txt\n", 24)
Source file: input.txt
D = 24
openat(AT_FDCWD, "input.txt", O_RDONLY) = 3
openat(AT_FDCWD, "copy_lowlevel.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644) = 4
lseek(3, 0, SEEK_SET) = 0
read(3, "This is a sample file used to de"..., 4096) = 157
write(4, "This is a sample file used to de"..., 157) = 157
read(3, "", 4096) = 0
close(3) = 0
close(4) = 0
write(1, "\n", 1) = 1
D = 1

```

```

openat(AT_FDCWD, "copy_standard.txt", O_WRONLY|O_CREAT|O_TRUNC, 0666) = 4
read(3, "This is a sample file used to de"... , 4096) = 157
read(3, "", 4096) = 0
read(3, "", 4096) = 0
close(3) = 0
write(4, "This is a sample file used to de"... , 157) = 157
close(4) = 0
write(1, "\n", 1) = 1
write(1, "Standard library I/O copy comple"... , 37Standard library I/O copy
completed.
) = 37
write(1, "Destination: copy_standard.txt\n", 31Destination: copy_standard.tx
t
) = 31
write(1, "Time: 0.011387825 seconds\n", 26Time: 0.011387825 seconds
) = 26
write(1, "\n", 1) = 1
write(1, "===== " , 45=====
) = 45
write(1, "PERFORMANCE COMPAR"... , 37PERFORMANCE
COMPARISON
) = 37
write(1, "===== " , 45=====
) = 45
write(1, "Low-level I/O : 0.009792089"... , 41Low-level I/O : 0.009
792089 seconds
) = 41
write(1, "Standard library : 0.011387825"... , 41Standard library : 0.011
387825 seconds
) = 41
write(1, "\n", 1) = 1
write(1, "Both copying methods completed s"... , 45Both copying methods compl
eted successfully.
) = 45
+++ exited with 0 +++
ruhi0404@Ruhitha:~/Operating_Systems/prac9task1$ |

```

Observation:

The file copy utility was successfully implemented using both low-level Linux I/O system calls and standard C library functions. The low-level implementation used `open()`, `read()`, `write()`, `lseek()`, and `close()`, while the standard library implementation used `fopen()`, `fread()`, `fwrite()`, and `fclose()`. The source file was successfully copied using both methods, and the generated files were verified against the original using file comparison commands. Execution time was measured for both approaches to demonstrate their performance differences. The system calls used by the low-level implementation were also verified using `strace`. Thus, file copying, correctness verification, system-call usage, and performance comparison between low-level and standard library I/O were successfully demonstrated.

Task-2: Develop a program that redirects standard input and Output using file descriptors and `dup2()`. Demonstrate how shells implement I/O redirection

```

GNU nano 7.2                                prac9task2.c *
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int input_fd;
    int output_fd;
    pid_t pid;
    int status;

    printf("=====\n");
    printf("          I/O REDIRECTION USING dup2()\n");
    printf("=====\n");

    /*
     * Open input file for reading.
     * This file will become standard input.
     */
    input_fd = open("input.txt", O_RDONLY);

    if (input_fd == -1)
    {
        perror("Error opening input.txt");
        return 1;
    }

    /*
     * Open output file for writing.
     * Create it if it does not exist.
     */
    output_fd = open("output.txt",
                     O_WRONLY | O_CREAT | O_TRUNC,
                     0644);

```

```

    /*
     * Open output file for writing.
     * Create it if it does not exist.
     */
    output_fd = open("output.txt",
                     O_WRONLY | O_CREAT | O_TRUNC,
                     0644);

    if (output_fd == -1)
    {
        perror("Error opening output.txt");
        close(input_fd);
        return 1;
    }

    printf("\nInput file descriptor : %d\n", input_fd);
    printf("Output file descriptor : %d\n", output_fd);

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        close(input_fd);
        close(output_fd);
        return 1;
    }

    if (pid == 0)
    {
        /*
         * Redirect standard input (fd 0)
         * to input.txt.
         */
        if (dup2(input_fd, STDIN_FILENO) == -1)
        {

```

```

    /*
     * Original file descriptors are no longer needed.
     */
    close(input_fd);
    close(output_fd);

    /*
     * Execute:
     * cat < input.txt > output.txt
     */
    execlp("cat", "cat", NULL);

    perror("execlp failed");
    exit(1);
}
else
{
    close(input_fd);
    close(output_fd);

    waitpid(pid, &status, 0);

    printf("\nChild process completed.\n");

    if (WIFEXITED(status) && WEXITSTATUS(status) == 0)
    {
        printf("I/O redirection completed successfully.\n");
    }
    else
    {
        printf("Child process failed.\n");
    }
}

return 0;
}

```

```

+++ exited with 0 +++
ruhi0404@Ruhitha:~/Operating_Systems/prac9task1$ cd ~/Operating_Systems
ruhi0404@Ruhitha:~/Operating_Systems$ mkdir -p prac9task2
ruhi0404@Ruhitha:~/Operating_Systems$ cd prac9task2
ruhi0404@Ruhitha:~/Operating_Systems/prac9task2$ pwd
/home/ruhi0404/Operating_Systems/prac9task2
ruhi0404@Ruhitha:~/Operating_Systems/prac9task2$ nano prac9task2.c
ruhi0404@Ruhitha:~/Operating_Systems/prac9task2$ nano input.txt
ruhi0404@Ruhitha:~/Operating_Systems/prac9task2$ gcc -Wall -Wextra prac9task
2.c -o prac9task2
ruhi0404@Ruhitha:~/Operating_Systems/prac9task2$ ./prac9task2
=====
      I/O REDIRECTION USING dup2()
=====

Input file descriptor : 3
Output file descriptor : 4

Child process completed.
I/O redirection completed successfully.
ruhi0404@Ruhitha:~/Operating_Systems/prac9task2$ cat output.txt
This is the input file.
The shell-style I/O redirection program will copy this content
from standard input to standard output using dup2().

ruhi0404@Ruhitha:~/Operating_Systems/prac9task2$ cat < input.txt > output.tx
t
ruhi0404@Ruhitha:~/Operating_Systems/prac9task2$ |

```

Observation: The program successfully demonstrated standard input and output redirection using file descriptors and the `dup2()` system call. An input file was opened and its file descriptor was redirected to standard input, while an output file descriptor

was redirected to standard output. A child process was created using `fork()` and the `cat` command was executed using `exec1p()`. The contents of the input file were successfully transferred to the output file, and the files were verified using `cmp`. The use of `dup2()` for redirecting file descriptors was also confirmed using `strace`. Thus, the practical successfully demonstrated the basic mechanism used by Linux shells to implement `<` and `>` I/O redirection.